

PopChoice — QuickBuild Guide

A reusable blueprint for rebuilding the PopChoice AI movie recommendation app from scratch, or adapting the same RAG architecture to another project.

Stack: React + Vite + Tailwind CSS + Lucide • Node.js + Express • Supabase PostgreSQL + pgvector • Hugging Face Transformers • Groq API

1. What PopChoice Does

PopChoice collects a user's movie-night preferences, converts them into a semantic query, creates an embedding, retrieves the most relevant movies from Supabase using pgvector, then gives those candidates to Groq. Groq selects one candidate and explains why it matches.

```
React / Vite
  ↓
Express API
  ↓
Build semantic query
  ↓
Hugging Face embedding
  ↓
Supabase + pgvector
  ↓
Top movie candidates
  ↓
Groq LLM / RAG
  ↓
Structured recommendation
  ↓
React MovieResult
```

2. Prerequisites

- Node.js 22.x. The working project used Node v22.23.2.
- npm 10.x. The working project used npm 10.9.8.
- Supabase account/project.
- Groq API key.
- Git and a code editor.
- On Windows, local ONNX inference may require the Microsoft Visual C++ Redistributable.

3. Project Structure

```
popchoice/
├── apps/
│   ├── api/
│   │   ├── src/
│   │   │   ├── config/supabase.js
│   │   │   ├── controllers/recommendation.controller.js
│   │   │   ├── routes/recommendation.routes.js
│   │   │   └── services/
│   │   │       ├── embedding.service.js
│   │   │       ├── groq.service.js
│   │   │       ├── recommendation.service.js
│   │   │       └── utils/buildRecommendationQuery.js
│   │   └── server.js
│   └── .env
├── package.json
└── web/
    ├── src/
    │   ├── assets/
    │   ├── components/
    │   │   ├── Brand.jsx
    │   │   ├── StartScreen.jsx
    │   │   ├── QuestionFlow.jsx
    │   │   ├── MovieResult.jsx
    │   │   └── LoadingScreen.jsx
    │   ├── data/movieOptions.js
    │   ├── lib/api.js
    │   ├── App.jsx
    │   ├── main.jsx
    │   └── index.css
    └── package.json
└── README.md
```

4. Frontend Setup

```
npm create vite@latest apps/web -- --template react
cd apps/web
npm install
npm install lucide-react
npm install tailwindcss @tailwindcss/vite
```

The frontend uses Vite + React JavaScript. Tailwind is imported through the current Tailwind/Vite setup. PopChoice also keeps a small amount of custom CSS for its desktop layout.

```
@import "tailwindcss";

@theme {
  --color-popdark-900: #0b0f19;
  --color-popdark-800: #111827;
  --color-popdark-700: #1f2937;
  --color-popgreen: #10b981;
  --color-popgreen-hover: #059669;
}
```

5. Backend Setup

```
mkdir apps/api
cd apps/api
npm init -y

npm install express cors dotenv @supabase/supabase-js openai
npm install @huggingface/transformers onnxruntime-node

npm install -D nodemon
```

The working backend used ES modules and these scripts:

```
{
  "type": "module",
  "scripts": {
    "dev": "nodemon src/server.js",
    "start": "node src/server.js"
  }
}
```

The Groq integration uses the OpenAI-compatible SDK package. The architecture does not depend on OpenRouter.

6. Installed Dependency Reference

Package	Role
react / react-dom	Frontend UI
vite	Frontend dev server/build
tailwindcss	Styling
@tailwindcss/vite	Tailwind Vite integration
lucide-react	Icons
express	Backend API
cors	Frontend → backend CORS
dotenv	Environment variables
@supabase/supabase-js	Supabase/PostgreSQL access
@huggingface/transformers	Local embedding model
onnxruntime-node	ONNX runtime for local embeddings
openai	OpenAI-compatible Groq client
nodemon	Backend development restart

7. Environment Variables

```
# apps/api/.env
```

```
PORT=5000
```

```
SUPABASE_URL=your_supabase_project_url  
SUPABASE_SERVICE_ROLE_KEY=your_supabase_service_role_key
```

```
GROQ_API_KEY=your_groq_api_key
```

Never put GROQ_API_KEY or the Supabase service-role key in the Vite frontend.

8. Supabase: Enable pgvector

In Supabase SQL Editor, enable the vector extension:

```
create extension if not exists vector;
```

9. Supabase: Create the Movies Table

```
create table if not exists movies (  
  id uuid primary key default gen_random_uuid(),  
  title text not null,  
  year integer,  
  description text,  
  genres text[] default '{}',  
  rating numeric,  
  poster_url text,  
  runtime_minutes integer,  
  embedding vector(384),  
  created_at timestamptz default now()  
);
```

The 384 dimensions are important. Xenova/all-MiniLM-L6-v2 produces 384-dimensional embeddings, so the pgvector column must match.

10. Insert Starter Movie Data

```
insert into movies  
(title, year, description, genres, rating, runtime_minutes)  
values  
(  
  'The Martian', 2015,  
  'An astronaut stranded on Mars must survive and find a way home.',  
  ARRAY['Sci-Fi', 'Adventure', 'Drama'], 8.0, 144),  
  
(  
  'The Shawshank Redemption', 1994,  
  'Two imprisoned men form a lasting friendship and find hope.',  
  ARRAY['Drama'], 9.3, 142),  
  
(  
  'Forrest Gump', 1994,  
  'A kind-hearted man experiences extraordinary events across decades.',  
  ARRAY['Drama', 'Romance'], 8.8, 142),  
  
(  
  'Interstellar', 2014,  
  'Explorers travel beyond the limits of space and time to save humanity.',  
  ARRAY['Sci-Fi', 'Adventure', 'Drama'], 8.7, 169),  
  
(  
  'The Pursuit of Happyness', 2006,  
  'A struggling father works to build a better life for his son.',  
  ARRAY['Drama', 'Biography'], 8.0, 117),  
  
(  
  'Back to the Future', 1985,  
  'A teenager travels through time and accidentally changes the past.',  
  ARRAY['Sci-Fi', 'Comedy', 'Adventure'], 8.5, 116);
```

11. Add Poster URLs

Poster URLs were stored in poster_url. In another project, use a licensed/stable image source.

```
update movies  
set poster_url = 'YOUR_POSTER_URL'  
where title = 'The Martian';
```

Repeat for the other movies.

12. Generate Movie Embeddings

The selected embedding model was Xenova/all-MiniLM-L6-v2. It runs locally through Transformers/ONNX and produces 384 values.

```
const text = `  
Title: ${movie.title}  
Description: ${movie.description}  
Genres: ${movie.genres.join(", ")}  
Year: ${movie.year}
```

```
Rating: ${movie.rating}
Runtime: ${movie.runtimeMinutes} minutes
`.trim();
```

```
const embedding = await createEmbedding(text);
```

The important rule is consistency: records stored in the vector database and future user queries must be embedded into the same vector space.

13. Embedding Service

```
import { pipeline } from "@huggingface/transformers";

let extractor;

async function getExtractor() {
  if (!extractor) {
    extractor = await pipeline(
      "feature-extraction",
      "Xenova/all-MiniLM-L6-v2",
    );
  }

  return extractor;
}

export async function createEmbedding(text) {
  const model = await getExtractor();

  const output = await model(text, {
    pooling: "mean",
    normalize: true,
  });

  return Array.from(output.data);
}
```

14. The Supabase Vector Search Query / RPC

This is the key SQL function used by the backend to retrieve semantically similar movies.

```
create or replace function match_movies(
  query_embedding vector(384),
  match_count int
)
returns table (
  id uuid,
  title text,
  year integer,
  description text,
  genres text[],
  rating numeric,
  poster_url text,
  runtime_minutes integer,
  similarity float
)
language sql
stable
as $$
select
  m.id,
  m.title,
  m.year,
  m.description,
  m.genres,
  m.rating,
  m.poster_url,
  m.runtime_minutes,
  1 - (m.embedding <=> query_embedding) as similarity
from movies m
where m.embedding is not null
order by m.embedding <=> query_embedding
limit match_count;
$$;
```

pgvector's <=> operator provides cosine distance. Ordering by distance gets the nearest records; converting it with 1 - distance gives a similarity-style value for the application.

15. Supabase Client

```
import { createClient } from "@supabase/supabase-js";

export const supabase = createClient(
```

```

process.env.SUPABASE_URL,
process.env.SUPABASE_SERVICE_ROLE_KEY,
);

```

16. Build the User's Semantic Query

```

export function buildRecommendationQuery(preferences) {
  const {
    numberOfPeople,
    duration,
    favoriteMovie,
    movieStyle,
    mood,
    strandedPerson,
  } = preferences;

  return `
I am choosing a movie for ${numberOfPeople || "a group of people"}.

Available viewing time: ${duration || "flexible"}.

A movie I already like is:
${favoriteMovie || "No specific favorite movie provided"}.

Preferred movie style:
${movieStyle || "Any style"}.

Desired mood:
${mood || "Any mood"}.

The movie should be suitable for:
${strandedPerson || "a general audience"}.

Find movies that are similar in themes, tone, genre,
emotional experience, and overall viewing preference.
`.trim();
}

```

This is the bridge between structured UI answers and semantic retrieval.

17. Retrieve Candidates

```

const query = buildRecommendationQuery(preferences);
const embedding = await createEmbedding(query);

const { data, error } = await supabase.rpc("match_movies", {
  query_embedding: embedding,
  match_count: 5,
});

if (error) throw error;

if (!data || data.length === 0) {
  throw new Error("No matching movies found");
}

```

At this point, the application has a controlled list of relevant candidates. Groq is not being used as the movie database.

18. Groq Setup

```

import OpenAI from "openai";

const groq = new OpenAI({
  apiKey: process.env.GROQ_API_KEY,
  baseURL: "https://api.groq.com/openai/v1",
});

```

The working model was openai/gpt-oss-20b. Keep the model configurable because provider model names can change.

19. Candidate Context for Groq

```

const candidateContext = candidates
  .map(
    (movie, index) => `
Candidate ${index + 1}:
ID: ${movie.id}
Title: ${movie.title}
Year: ${movie.year}
Description: ${movie.description}
Genres: ${movie.genres.join(", ")}
Rating: ${movie.rating}
Runtime: ${movie.runtimeMinutes} minutes
Semantic similarity: ${movie.similarity}

```

```
,
)
.join("\n");
```

20. RAG Prompt Rules

You are PopChoice, an AI movie recommendation assistant.

IMPORTANT RULES:

1. You may ONLY recommend a movie from the provided candidates.
2. Never invent a movie.
3. Never modify a movie title.
4. Consider the user's mood, preferred style, favorite movie, available viewing time, and audience.
5. Semantic similarity is useful evidence but is not the only deciding factor.
6. Return valid JSON only.

The model is asked to return:

```
{
  "movieId": "candidate movie id",
  "title": "movie title",
  "reason": "Why this movie fits",
  "matchHighlights": [
    "specific reason",
    "specific reason",
    "specific reason"
  ]
}
```

21. Validate the Groq Result

```
const selectedMovie = candidates.find(
  (movie) => movie.id === recommendation.movieId,
);

if (!selectedMovie) {
  throw new Error(
    "Groq selected a movie that was not in the retrieved candidates",
  );
}
```

This is an important RAG control: the model can reason about your data, but it cannot introduce a movie that your retrieval layer did not provide.

22. Recommendation Service Flow

```
export async function generateRecommendation(preferences) {
  const candidates = await findMovieCandidates(preferences);

  const recommendation = await generateMovieRecommendation(
    preferences,
    candidates,
  );

  const selectedMovie = candidates.find(
    (movie) => movie.id === recommendation.movieId,
  );

  if (!selectedMovie) {
    throw new Error(
      "Groq selected a movie that was not in the retrieved candidates",
    );
  }

  return {
    movie: selectedMovie,
    reason: recommendation.reason,
    matchHighlights: recommendation.matchHighlights,
  };
}
```

23. Express Controller and Route

```
// recommendation.routes.js
import { Router } from "express";
import { createRecommendation } from "../controllers/recommendation.controller.js";

const router = Router();

router.post("/", createRecommendation);
```

```

export default router;

// recommendation.controller.js
export async function createRecommendation(req, res) {
  try {
    const preferences = req.body;

    if (!preferences || typeof preferences !== "object") {
      return res.status(400).json({
        success: false,
        message: "Preferences are required",
      });
    }

    const recommendation =
      await generateRecommendation(preferences);

    return res.status(200).json({
      success: true,
      data: recommendation,
    });
  } catch (error) {
    console.error("Recommendation error:", error);

    return res.status(500).json({
      success: false,
      message: "Failed to generate recommendation",
    });
  }
}

```

24. Express Server

```

import "dotenv/config";
import express from "express";
import cors from "cors";

import recommendationRoutes from "../routes/recommendation.routes.js";

const app = express();
const PORT = process.env.PORT || 5000;

app.use(
  cors({
    origin: "http://localhost:5173",
  })
);

app.use(express.json());

app.get("/api/health", (req, res) => {
  res.json({
    success: true,
    message: "PopChoice API is running",
  });
});

app.use("/api/recommendations", recommendationRoutes);

app.listen(PORT, () => {
  console.log(
    `PopChoice API running on http://localhost:${PORT}`
  );
});

```

25. Frontend API Connection

```

const API_URL = "http://localhost:5000/api";

export async function getRecommendation(preferences) {
  const response = await fetch(
    `${API_URL}/recommendations`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(preferences),
    }
  );

  const data = await response.json();

  if (!response.ok) {
    throw new Error(
      data.message || "Failed to get recommendation",
    );
  }
}

```

```

    });
  }

  return data.data;
}

```

26. Frontend Preference Shape

```

{
  numberOfPeople: 2,
  duration: "2 hours",
  favoriteMovie: "The Shawshank Redemption",
  movieStyle: "Classic",
  mood: "Inspiring",
  strandedPerson: "Morgan Freeman"
}

```

27. Frontend Flow

```

StartScreen
  ↓
numberOfPeople + duration
  ↓
QuestionFlow
  ↓
favoriteMovie + movieStyle + mood + strandedPerson
  ↓
POST /api/recommendations
  ↓
Loading state
  ↓
MovieResult
  ↓
Try Again → StartScreen

```

- StartScreen validates the first two fields.
- QuestionFlow validates all four remaining fields.
- App.jsx combines the preference objects.
- The API request runs only after validation passes.
- Loading and error states are shown while the backend works.
- MovieResult displays the returned movie and Groq explanation.

28. Test with Postman First

POST http://localhost:5000/api/recommendations

Content-Type: application/json

```

{
  "numberOfPeople": 2,
  "duration": "2 hours",
  "favoriteMovie": "The Shawshank Redemption",
  "movieStyle": "Classic",
  "mood": "Inspiring",
  "strandedPerson": "Morgan Freeman"
}

```

The working backend was verified in Postman before connecting the redesigned UI. This separated backend/RAG problems from frontend problems.

29. Useful Test Cases

```

// Fun / classic
{
  "numberOfPeople": 4,
  "duration": "2 hours",
  "favoriteMovie": "Back to the Future",
  "movieStyle": "New",
  "mood": "Fun",
  "strandedPerson": "Chris Pratt"
}

// Serious
{
  "numberOfPeople": 2,
  "duration": "2 hours",
  "favoriteMovie": "The Pursuit of Happyness",

```

```

    "movieStyle": "Classic",
    "mood": "Serious",
    "strandedPerson": "Denzel Washington"
  }

  // Sci-fi / inspiring
  {
    "numberOfPeople": 3,
    "duration": "3 hours",
    "favoriteMovie": "Interstellar",
    "movieStyle": "New",
    "mood": "Inspiring",
    "strandedPerson": "Matthew McConaughey"
  }

```

30. Problems We Encountered and Fixes

- Browser-side AI code caused process/environment issues → AI calls and secrets were moved to Express.
- The frontend initially used a mock recommendation flow → it was replaced with the real API request.
- ONNX local inference failed on Windows → the Visual C++ runtime was installed.
- Embedding dimensions had to match → pgvector was set to vector(384).
- Groq had a token-limit issue in an earlier AI setup → prompt/candidate context and output requirements were controlled.
- The LLM could theoretically invent a movie → it was restricted to retrieved candidates and the returned ID is validated.
- The original Figma-style implementation was mobile-sized → StartScreen and QuestionFlow were redesigned for responsive desktop/web use.
- Old CSS accumulated during iterations → unused mobile/Figma CSS can be removed once the final JSX no longer references it.

31. Rebuild Order — The Fastest Path

PHASE 1 — FOUNDATION

1. Create React/Vite frontend.
2. Create Express backend.
3. Install dependencies.
4. Configure .env.

PHASE 2 — DATA

5. Create Supabase project.
6. Enable pgvector.
7. Create table.
8. Insert records.
9. Create match RPC.

PHASE 3 — EMBEDDINGS

10. Install Transformers + ONNX runtime.
11. Load all-MiniLM-L6-v2.
12. Embed your records.
13. Store vectors.

PHASE 4 — RETRIEVAL

14. Build semantic query from user input.
15. Embed the query.
16. Call match_records / match_movies.
17. Inspect the returned candidates.

PHASE 5 — RAG

18. Configure Groq.
19. Build candidate context.
20. Write a constrained JSON prompt.
21. Parse and validate the result.

PHASE 6 — API

22. Add service.
23. Add controller.
24. Add route.
25. Test in Postman.

PHASE 7 — UI

26. Build forms.
27. Add validation.
28. Connect fetch API.
29. Add loading/error states.
30. Render the result.

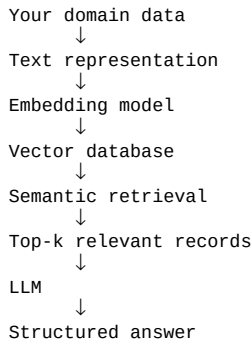
PHASE 8 — PRODUCTION

31. Secure secrets.

- 32. Add rate limiting.
- 33. Add request validation.
- 34. Add logging/monitoring.
- 35. Deploy frontend and API.
- 36. Configure production CORS.
- 37. Add indexes/caching as the dataset grows.

32. Reuse This for Another AI Project

The movie theme is not the important part. The reusable architecture is:



- For books: retrieve relevant books/chapters.
- For jobs: retrieve matching job descriptions/candidate profiles.
- For products: retrieve products based on user requirements.
- For documentation: retrieve relevant documentation chunks.
- For customer support: retrieve relevant knowledge-base articles.
- For travel: retrieve destinations, activities, and constraints.

To adapt it, change the database schema, embedding text, semantic query builder, prompt, and output schema. The retrieval → generation pattern stays the same.

33. Production Checklist

- Move API URLs into environment variables.
- Never expose Groq or Supabase service-role secrets in the browser.
- Validate request bodies with a schema validator.
- Add rate limiting.
- Log embedding, retrieval, and LLM latency separately.
- Consider caching repeated semantic queries.
- Add vector indexes when the dataset becomes large.
- Make the model name configurable.
- Use stable/licensed poster and media URLs.
- Add tests for the Supabase RPC, recommendation service, controller, and frontend flow.
- Use a production CORS origin instead of localhost.

34. Final Mental Model

- **React** — collects preferences and displays the recommendation.
- **Express** — protects secrets and orchestrates the workflow.
- **Embedding model** — converts text into semantic vectors.
- **Supabase + pgvector** — retrieves relevant records.
- **Groq** — reasons over retrieved records and produces the explanation.
- **RAG** — grounds generation in your own retrieved data instead of treating the LLM as your database.

35. Quick Command Reference

```
# Frontend
cd apps/web
npm install
npm run dev

# Backend
cd apps/api
npm install
npm run dev

# Backend health check
GET http://localhost:5000/api/health

# Recommendation
POST http://localhost:5000/api/recommendations
```

36. Important Version Note

The working PopChoice environment used `@huggingface/transformers 4.3.0`, `onnxruntime-node 1.30.0`, and Groq model `openai/gpt-oss-20b`. Dependency versions and model names can change. When rebuilding later, use compatible current versions while preserving the architectural contracts: matching embedding dimensions, vector retrieval, server-side secrets, constrained RAG output, and validation of the selected record.

End of QuickBuild Guide

If you can rebuild the pipeline in this order — database → embeddings → vector search → Groq RAG → API → frontend — you can reuse the same architecture for many AI applications.